General DIA peak-memory: columnar (StructArray) scored-PSM storage

Eliminate the scored-PSM extraction copy on every DIA run — plan + code + validation

Pioneer.jl branch `feat/zt-batched-collapse`

2026-07-21

Abstract

On EVERY DIA search (ZT and non-ZT alike) the post-deconvolution raw PSMs are materialized in ∼3 copies at the Main-Search peak: (1) the per-thread `Vector{MainSearchScoredPSM}` buffer `Score!` writes into (measured 6.3 GB/10 threads on 5 min ZT; the *only* large deconv buffer — `Hs_fused`/id-maps/working vectors are < 0.01 GB each), (2) the `DataFrame(@view(scored[1:last_val]))` each thread returns — Tables.jl extracts the struct fields into fresh column vectors, a *full copy*, and (3) the `vcat(fetched...)` concatenation. This plan removes copy (2), the extraction, by storing the scored buffer as a `StructArray` (fields already laid out as separate column vectors), so the returned `DataFrame` *wraps* those columns instead of copying them. Byte-identical (same values, different memory layout); benefits every DIA run; no disk IO. The change is nearly a one-liner (the storage type), but carries two things to verify carefully: hot-loop `setindex!` performance and column-aliasing safety.

1 Why this is general (not ZT-specific)

The path is shared: `process_scans_fused!` (`process_scans_fused.jl:176`) returns `DataFrame(@view(get_scored_psms` for *all* search methods, and `library_search` vcats them. Astral/Exploris (∼48M PSMs) pay a *larger* version of the same triple-buffering. ZT additionally has the collapse (raw→meta) that the disk-spill exploits; this optimization is orthogonal and stacks with it.

2 The change (minimal core)

`main_search_scored_psms::Vector{MainSearchScoredPSM{Float32,Float16}}` → `::StructArray{MainSearchScoredPSI` (same element type). Everything the code does to it already has a `StructArray` method:

site	operation	StructArray?
`ScoredPSMs.jl Score!`	`scored[i] = MainSearchScoredPSM(...)`	yes (splats struct into columns)
`ScoredPSMs.jl growScoredPSMs!`	`resize!(scored, n)`	yes (resizes every column)
`SearchMethods.jl:284`	allocate `(undef, 5000)`	`StructArray{T}(undef, 5000)`
`process_scans_fused.jl:176`	`DataFrame(@view(scored[1:last_val]))`	**now wraps columns zero-copy**

The same return line becomes zero-copy: `@view(structarray[1:n])` is a `StructArray` of column *views*, and `DataFrame` of it wraps those views (Tables.jl column access) instead of extracting. Also flip `TuningScoredPSM` storage the same way (tuning paths).

2.1 Zero-copy conversion (explicit, if the implicit wrap needs forcing)

```
# get_scored_psms(...) now returns a StructArray. Wrap the first last_val rows of each
# component column as a view -> DataFrame holds SubArray columns, no extraction copy.
function scored_view_df(sa::StructArray, last_val::Int)
    cols = StructArrays.components(sa)                  # NamedTuple of column Vectors
    DataFrame(Any[view(c, 1:last_val) for c in cols], collect(keys(cols)); copycols=false)
end
# process_scans_fused.jl:176 becomes:
return scored_view_df(get_scored_psms(search_data, params), last_val)
```

3 Correctness (byte-identity)

A `StructArray{T}` stores exactly the same field values as `Vector{T}` — only the memory layout differs (SoA vs AoS). `Score!` writes identical values; the DataFrame exposes identical columns. So results are bit-identical. **Aliasing safety** (the one real hazard): the returned DataFrame's columns are now *views* into the scratch buffer, so we must ensure nothing mutates them in place or frees the buffer while a view is live:

- **Non-ZT:** the per-thread DataFrame is only an input to `vcat(fetched...)`, which *copies*; `scan_competition/ms1/permute` run on the vcat'd copy, never on the views. Safe. Next file, `Score!` overwrites the buffer — but the vcat'd table is an independent copy. Safe.
- **ZT disk-spill:** `scan_competition/ms1` only *append* columns (never mutate `:weight`/`:scan_idx`); `writeArrow` reads and copies to disk; only *then* is the buffer freed (`tbl=nothing` first, so no live view). Safe.
- **Guard:** add an assertion/test that no pass mutates an existing scored column in place before vcat/scatter (they don't today).

4 Risks to measure (this is where the care goes)

1. **Hot-loop `setindex!` performance.** `Score!` does ∼33M struct assignments. `StructArray setindex!` destructures the struct and writes each component — should be alloc-free and type-stable, but *must be benchmarked*: compare Main-Search deconv time before/after. If a regression appears, check `@code_warntype` on `Score!` and that the `StructArray` component types are concrete.
2. **DataFrame-view semantics.** Confirm `DataFrame(@view(sa[1:n]))` does not silently copy (check allocation) and that `copycols=false` holds through `vcat`.
3. **Type stability of the field.** `StructArray{MainSearchScoredPSM{Float32, Float16}}` has a concrete, fully-specified component `NamedTuple` type — verify no `Any` creeps into `getMainSearchScoredPsms` call sites.
4. **Downstream that indexes the buffer as structs** (e.g. `process_scans.jl:191`, any `scored[i].field` access) — `StructArray` supports struct-style indexing, but audit every read site.

5 Benchmark + validation protocol

All runs single-file 5 min ZT first (fast, deterministic), then a non-ZT set.

1. **Correctness gate:** byte-identical to current: ZT `2,950,447` meta / `26,167` prec / `4,213` PG (spill on AND off); metascan dump content-identical to `meta_global`. Non-ZT (Astral/Exploris) result bit-identical.
2. **Hot-loop speed:** Main-Search deconv time (report's "Main Search" + the `library_search deconv` breakdown) before/after — the primary risk metric. Use the warm driver (`warm_driver.jl`) so JIT is hot; compare like-for-like.
3. **Peak / min-memory:** `PIONEER_DIAG_RSS` sub-step markers + `time -l phys_footprint`; and the `--heap-size-hint=NG` min-memory test (does removing the extraction copy let it run at a lower hint than today?).
4. **Allocation:** `PIONEER_DIAG_ALLOC library_search` bucket should drop by ∼one raw-table's worth (∼6 GB cumulative) if the extraction copy is gone.

6 Rollout

- Land behind a flag first (`PIONEER_SCORED_STRUCTARRAY=1`) if we want an A/B during validation; otherwise it is byte-identical and can replace the storage directly once the hot-loop benchmark is clean.
- Order: (a) flip `MainSearchScoredPSM` storage + conversion; validate ZT byte-identity + deconv speed. (b) flip `TuningScoredPSM`. (c) non-ZT byte-identity. (d) measure the min-memory (heap-size-hint) improvement, ZT and non-ZT.
- Composes with the disk-spill: spill removes the vcat and holds one bin; StructArray removes the extraction copy. Together the raw PSMs approach a single columnar copy.

7 Open question for review

Removing copy (2) leaves copies (1) the SoA buffer and (3) the vcat. For non-ZT, (3) is still needed (global per-precursor passes span round-robin threads). Is a follow-up worth it to also drop (3) — e.g. a pre-sized shared columnar buffer (two-pass count-then-fill) so threads write into one table with no vcat? That would leave a single columnar copy of the raw PSMs for all runs. Bigger change; flagged for later.